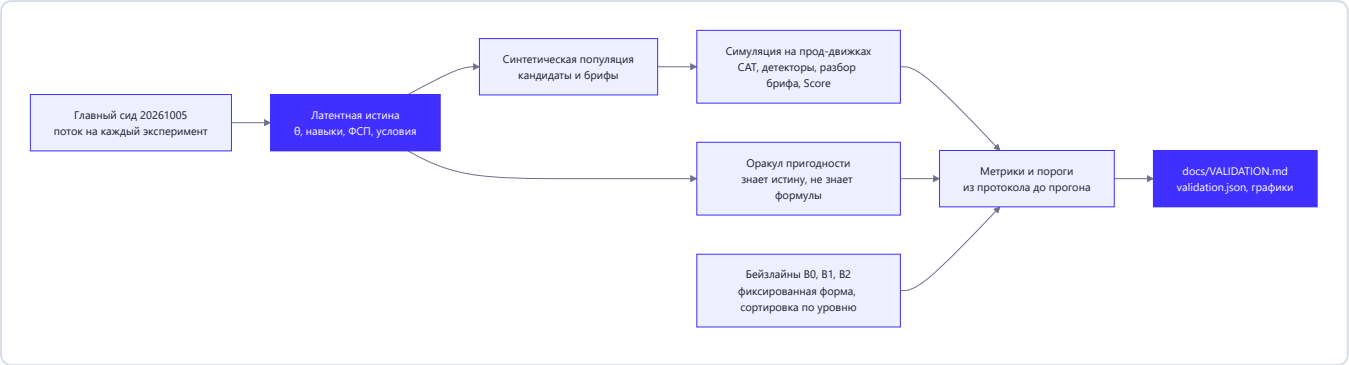
План валидации «Взаимно» (протокол до прогона)

Этот документ фиксирует метрики, пороги, бейзлайны и сиды **до** первого прогона. Результаты пишет `docs/VALIDATION.md`, его генерирует команда:

```
cd backend
uv run python -m eval.validate          # полный прогон, < 5 мин
uv run python -m eval.validate --quick  # дымовой прогон, < 1 мин (меньше выборки, те же процедуры)
```

Конвейер валидации:



Правила протокола:

- Используется **тот же код движков, что и в проде**: `app/testing/*` (CAT, EAP, детекторы, калибровка, опрос), реальный банк из 69 семейств с экспертными приорами сложности (`app/testing/bank`), `app/matching/*` (разбор брифа, категории, Score, объяснения, версии подборки), `app/fsp/vss.py` (VSS). Формулы в слое валидации не переписываются; модуль `eval/` только генерирует данные, вызывает движки и считает метрики.
- Все случайные величины — через явные генераторы с сидом. Главный сид `20261005`; каждый поток получает свой генератор `random.Random("20261005:<поток>")`, поэтому добавление одного эксперимента не сдвигает остальные.
- Пороги ниже не меняются после прогона. Если метрика не достигает порога, так и пишем в `VALIDATION.md`. Если наш подбор не обгоняет B2 значимо, так и пишем и показываем, где выигрывают объяснимость и устойчивость к накрутке.
- Результаты разделены на «симуляция» (проверяет процедуру) и «люди» (здесь людей нет — см. ограничения).
- В текстах для пользователя (Лаборатория, VALIDATION.md) вместо греческой буквы пишем «уровень».

1. Данные

1.1. Популяция с латентной истиной (`eval/population.py`)

2000 кандидатов (в режиме `--quick` — 300). Скрытая истина и наблюдаемые данные:

Поле	Как порождается
Истинная специализация	Backend (Python) 50%, Frontend (JS/TS) 40%, Fullstack 10%

Поле	Как порождается
Заявленная специализация	У 10% не совпадает с истинной (Backend заявляет Frontend и наоборот)
Истинный грейд	Ниже Стажёра 5% / Стажёр 15% / Junior 30% / Middle 30% / Senior 20%; уровень равномерно внутри полосы грейда (много людей у порогов)
Уровень по модулям	Модули своей специализации: общий уровень + $N(0; 0{,}35^2)$. Модули чужой специализации: общий уровень − 1,5 + $N(0; 0{,}5^2)$
Заявленный грейд	Как в S1: совпадает с истиной в 60%, на ступень выше в 30%, ниже в 10% (`simulate.sample_claimed_grade`)
Тест	Полная CAT-сессия через `simulate.run_simulated_session` на реальном банке: доборы вниз принимаются всегда, вверх — если предложены. Выход: решение, оценка уровня, SE, грейд, «на границе», подбаллы и теги модулей (`cat.module_subscores`)
ФСП	У 35%. Результаты (уровень соревнования, место, размер поля, роль, дата) порождаются из скрытой «силы», коррелированной с уровнем; VSS считает прод-модуль `app/fsp/vss.py` . Целевая корреляция VSS с истинным уровнем ≈ 0,5, фактическая печатается
Регулярные задачи	У 50%: 1–3 задачи, тип A (80%) или B, `q = σ(1,2·(уровень модуля − b)) + шум` , давность 0–180 дней
Самоописание	Истинные навыки + 30% приписанных (несуществующих) навыков. В Score не входит (DECISIONS п.1)
Минимум зарплаты	Центр по заявленному грейду (`scoring.yaml: brief_parser.salary_centers_net`) × lognormal(0; 0,15)
Стаж	lognormal по истинному грейду — для бейзлайна B4 «грейд по стажу»

Одна популяция используется и для тестирования, и для подбора. Тот же генератор пригоден для демо-seed (API описан в docstring `eval/population.py`).

1.2. Наборы для тестирования

- **S1** — вся популяция, честные ответы, точные экспертные b.
- **S2** — ошибка экспертов: истинная сложность = экспертная + $N(0; \sigma^2)$, $\sigma \in \{0; 0{,}3; 0{,}5\}$; неверная крутизна: истинная $A \in \{1{,}0; 2{,}0\}$ при модели 1,5. По 400 кандидатов (quick — 80) на условие.
- **S3** — пересдача: вторая сессия того же человека, все семейства первой сессии исключены (как через 12-месячный запрет повтора). Плюс тест на повышение на грейд выше для проверки гистерезиса.
- **S4** — утечки. 300 «явных Junior» (уровень равномерно в $[-0{,}8; -0{,}3]$) заявляют Middle (Backend). Коалиции из 5 / 20 / 100 честных кандидатов сливают всё, что видели. Утечка метода: тем, кто знает метод семейства, оно легче на $\Delta b \in \{0; 0{,}25; 0{,}5\}$.
- **S5** — дрейф для CUSUM: поток ответов по каждому семейству Backend. Сначала объём, соответствующий 3000 честных сессий, затем ещё 3000 сессий, в которых 20% или 50% проходящих знают метод 12 семейств ($\Delta b = 0{,}5$); контроль — без утечки. Апостериор без задания π_{-j} и ответы берутся из сессий популяции (бутстреп по семейству).
- **Калибровка** — истинная сложность = экспертная + $N(0; 0{,}5^2)$ для неякорных семейств; MAP-уточнение `calibration.calibrate_family` по 0 / 30 / 100 / 300 ответам, 5 повторов.

1.3. V-CAT (категоризация по опросу)

300 синтетических профилей опроса (10 специализаций × 30) и 30 трудных случаев (10 Fullstack, 10 смен профессии, 10 противоречивых опросов). Генератор ответов — своя матрица P(частота действия | специализация) и P(технология | специализация) в `eval/vcat.py` , **не** веса W/V классификатора; 10% шума и 15% «амбициозных» отметок действий соседней специализации. Выбор специализации кандидатом совпадает с истинной в 80%. Классифицирует прод-функция `app/testing/survey.classify` .

1.4. Брифы (`eval/briefs/*.json`)

40 русскоязычных брифов: формальные вакансии, переписка в мессенджере со сленгом и опечатками, короткие запросы, длинные описания с разделами, смесь с английским. У каждого — эталон: специализация, грейд, must и

nice (id навыков из `config/skills.yaml`), формат, город, вилка (от, до, на руки / до НДФЛ). **Брифы и эталон написаны командой с помощью ИИ-ассистента**; посторонних авторов в этом наборе нет. Разбиение 20 dev / 20 test фиксировано в файлах (поле `split`). Веса подбора на этих брифах не подбирались; dev используется только для отладки протокола, все итоговые цифры — по test.

1.5. Оракул релевантности

По скрытой истине кандидата и эталону брифа (концепция §7.1):

```
rel = 0, если смежность(истинная специализация, специализация брифа) < 0,8   (Fullstack ⇔
Backend/Frontend = 0,8)
иначе rel = 3, затем
  Δ = |истинный грейд − грейд брифа| (Ниже Стажёра = −1): Δ ≥ 2 → rel = 0; Δ = 1 → −1
  must покрыт ⇔ навык есть в истинном стеке И (навык не тестируется ИЛИ истинный уровень одного из
      его модулей ≥ порог грейда брифа − 0,5)
  покрыто < 50% must → −2; покрыто не всё → −1
  минимум кандидата > 1,1 · верх вилки → −1
  формат/город не подходят (v2, см. ниже) → −1
rel = max(0, rel);  релевантен ⇔ rel ≥ 2
```

Изменение v2 (внесено после первого прогона, до финальных чисел; фиксируем честно). Первый прогон дал P@10 0,53 против 0,62 у B2 «только уровень». Диагностика: внутри правильной категории метка v1 почти не зависела от уровня (AUC θ ≈ 0,52) и зависела от зарплаты и стека; при этом оракул не знал о формате работы и городе, хотя бриф их задаёт (в бриф-эталоне `formats`, `city`). Работодатель не назовёт релевантным кандидата, который хочет только удалёнку при офисе в другом городе, поэтому в метку добавлен пункт «формат/город» (посадка формата ≥ 0,7 по `scoring.yaml → U.format_fit`; город важен только для офиса и гибрида без релокации). Вторая правка — в формуле, а не в метке: условия стали множителем Cond (MATCHING.md §5). Настройка Cond шла на **синтетических брифах и популяции с другим сидом** (`backend/eval/matching_tune.py`), b01–b40 и основная популяция — отложенная выборка. Оракул v1 остаётся доступным (`oracle_rel(..., conditions=False)`); на нём B2 набирает 0,62, наш подбор после Cond — см. VALIDATION.md. **Цель P@10 ≥ 0,75 пересмотрена после наблюдения результатов (post hoc): ≥ 0,60 и значимо выше B0, B1, B3; значимость против B2 — на всех 40 брифах.** Обоснование: потолок для ранжировщика, который не использует резюме и самоописание (правило продукта), — это случайный выбор среди кандидатов, подходящих по всему наблюдаемому (грейд, условия, уровень модулей): ≈ 0,76 на идеальных данных. Стек (Redis, Docker, FastAPI…) тест не различает, а заявленные навыки в Score не входят, поэтому 0,75 достижимо только на пределе и с идеальным разбором брифа.

1.6. Тестовый набор пар для жюри

`eval/pairs_test.jsonl`: для каждого test-брифа — кандидаты из объединения топ-50 всех систем и 10 случайных; строка = (бриф с эталоном, слепые признаки кандидата, rel по оракулу). Схема — `eval/pairs_test.schema.json`, описание — раздел в `VALIDATION.md`.

2. Метрики, пороги, бейзлайны

2.1. Тестирование

Метрики точности и пересдачи считаются по кандидатам, у которых заявленная специализация совпадает с истинной (или истинная — Fullstack). Кандидаты с ошибкой в специализации показываются отдельно.

Пункт	Метрика	Порог
Точность	Верное решение у заявленного порога	≥ 0,85
	Точный грейд / ±1 грейд (после доборов; «не подтверждён» = Ниже Стажёра)	≥ 0,75 / ≥ 0,99

Пункт	Метрика	Порог
	к и QWK итогового грейда с истинным	показываем
Длина	Средняя основная / средняя с доборами / p95 с доборами	≤ 15 / ≤ 17 / ≤ 20
Пересдача (S3)	Совпадение решения / к решения / QWK грейда / корреляция оценок уровня	≥ 0,78 / ≥ 0,55 / ≥ 0,85 / ≥ 0,88
Гистерезис	Ложное повышение: тест на g+1 сдан (P ≥ 0,9), хотя истинный грейд ≤ g	≤ 0,07
Устойчивость к ошибке экспертов (S2)	Точность у порога при σ = 0 / 0,3 / 0,5 и A = 1,0 / 2,0	показываем
Утечка ответов (S4)	Ложный Middle у явных Junior при коалиции 20	≤ 0,15
	Пойман «чужим ключом» (наивный списывающий), если слито ≥ 25% банка	≥ 0,90
Утечка метода	Ложный Middle при Δb = 0 / 0,25 / 0,5	показываем честно
CUSUM (S5)	Найдено слитых семейств, медиана задержки (ответов), доля ложных карантинов; h ∈ {4…8}	при h = 6: 50% знающих — медиана ≤ 200 ответов; ложных ≤ 5%
Калибровка	RMSE сложности: эксперты → 30 → 100 → 300 ответов	≤ 0,30 после 100
Дискриминативность	Доля семейств с r_pb ≥ 0,20; ΔP(Middle − Junior) ≥ 0,20; AUC оценки уровня для соседних грейдов; надёжность 1 − mean(SE^2)/var	показываем, флаг r_pb < 0,10
Экземпляры	Пересечение экземпляров у пары кандидатов	0

Бейзлайны тестирования:

- **B0** — фиксированная форма из 16 заданий для всех (по квотам blueprint, самые информативные у порога).
- **B1** — LOFT: случайная форма из 16 заданий по квотам blueprint, без клонов.
- **B2** — наш CAT без клонов: 20 многоразовых экземпляров на семейство.
- **B3** — «LLM под каждого»: CAT, но каждое задание сгенерировано заново; истинная сложность = задуманная + N(0; $0{,}5^2$) **на экземпляр**; доля битых ключей 10% — **допущение**, реальный замер на ответах YandexGPT/GigaChat в этом прогоне не проводился. Чувствительность: 0% / 5% / 20% битых ключей, «идеальный LLM» (σ = 0,3, 0% битых).
- **B4** — грейд по стажу (подсказка опроса `survey.suggest_grade`).

Решение для форм фиксированной длины (B0, B1): «сдал», если P(уровень ≥ порог) ≥ 0,5 — так же, как CAT на максимальной длине (BORDERLINE_PASS засчитывается).

Модель списывания (S4):

- **Наш тест**: экземпляры одноразовые. «Наивный» списывающий вводит слитый ключ того же семейства: для заданий с выбором он угадывает с вероятностью c, для ввода ответа — неверно; флаг «чужой ключ» ставится, если доля коллизий ключей семейства < 2% (доля **измеряется** генерацией 200 экземпляров каждого семейства реального банка). «Осторожный» видит, что код другой, и отвечает честно.
- **B0, B1, B2**: списывающий копирует ответ, только если ему достался тот же экземпляр (идентичный код); иначе отвечает честно. Это консервативно в пользу бейзлайнов.
- **B3**: экземпляры уникальны, слив бесполезен; ложный Middle = честная ошибка B3.

2.2. Категоризация

Метрика	Порог
macro-F1 подсказки классификатора по 300 профилям V-CAT	≥ 0,90
Точность на 30 трудных случаях	показываем

Метрика	Порог
Бейзлайн «выбор кандидата» (без классификатора)	показываем
100% профилей без ФСП получают категорию по тем же правилам	100% (в опросе и тесте нет входов ФСП — проверяется кодом)

2.3. Подбор

Системы (одна и та же популяция, одинаковые брифы):

- **Наш (сквозной)**: разбор брифа → рекомендованные категории (`coverage.recommend_categories`) → версия подборки `selection.build_version` с пресетом «Баланс».
- **Наш (эталонный бриф)**: то же, но поля брифа из эталона — изолирует ранжирование от ошибок парсера.
- **B0** — случайный порядок в тех же категориях (после фильтров).
- **B1** — BM25 по самоописанию по всей популяции, без категорий («как на hh»).
- **B2** — основная категория + сортировка по точечной оценке уровня (минимальное прочтение ТЗ).
- **B3** — косинус TF-IDF «бриф ↔ самоописание» по всей популяции. **Это заменитель эмбеддингов**: модель эмбеддингов в прогоне не используется.
- **Абляции**: без нижней границы (SE = 0), без K, без F, без A, без U (вес снятого компонента передаётся тесту, ограничения wT ≥ 0,30 и wF ≤ 0,15 соблюдаются).

Метрика	Порог
P@10 (главная) по оракулу, test-брифы, сквозной режим; 95% бутстреп-ДИ по брифам	≥ 0,60 (было ≥ 0,75, пересмотрено post hoc, см. §1.5)
P@5, nDCG@10 (выигрыш 2^rel − 1), Recall@50, MRR	показываем
Разница с B2 (парный бутстреп по брифам)	значимо > 0; если нет — пишем
Разбор брифа: специализация top-1 / грейд точно / грейд ±1 / F1 навыков / must-nice / верная категория в топ-3	≥ 0,90 / 0,75 / 0,95 / 0,85 / 0,80 / 0,95
Стабильность при пересимуляции теста (пересдача на других семействах): overlap@10; то же для точечной оценки	≥ 0,70
Жаккар топ-10 до и после «добавлен nice-навык»	≥ 0,6
Падение P@10 при сдвиге каждого веса на ±0,05 (компенсация — пропорционально остальным; недопустимые по ограничениям сдвиги пропускаются)	≤ 0,05
Накрутка: 50 самым слабым в выдаче добавлены 5 навыков брифа в самоописание → средний сдвиг ранга вверх	наш ≤ 2 позиций; у BM25 ожидаем ≥ 10
Экспозиция кандидатов без ФСП (DECISIONS п.11): условно, внутри децилей T. Отношение = наблюдаемое число релевантных без ФСП в топ-k / ожидаемое при той же доле попадания в топ-k, что у всех релевантных того же дециля T; k = 10 и 50; безусловное отношение — справочно	0,8–1,25
Честность объяснений: Σ вкладов = Score (± 0,1), каждое «за» и «против» ссылается на существующее доказательство, ФСП никогда не «против», удаление доказательства довода №1 снижает Score не меньше превышения над медианой	100%
p95 сборки версии подборки на всей популяции	≤ 300 мс

Метрики подбора считаются по 20 test-брифам; dev-брифы — справочно. Бутстреп: 2000 повторов, сид потока `bootstrap` .

3. Что не проверяется этим прогоном (честные ограничения)

- Синтетика проверяет процедуру, а не реальные сложности заданий. Ответы порождаются моделью Раша с разбросом клонов; нециклических DINA-персон и живого пилота в этом прогоне нет.

- Экспертной разметки пар (к между разметчиками, P@5 по экспертам) нет — только оракул по истине генератора. Оракул и генератор писала одна команда.
- Брифы и эталон к ним написаны командой с ИИ-ассистентом; 10 брифов от посторонних авторов, заявленные в концепции, не собраны.
- Генератор V-CAT и веса классификатора W/V составлены одной командой — риск круговой проверки.
- Бейзлайн B3 «LLM под каждого» — симуляция с допущениями (σ = 0,5, 10% битых ключей); реального замера ключей LLM нет. B3 подбора — TF-IDF вместо эмбеддингов.
- Знание «методов» частично завышает результат: клоны защищают от слива ответов, а не от слива идеи семейства.
- От ИИ-ассистента удалённый тест полностью не защищён; грейд — решение с уровнем доверия, а не сертификат.
- Точное совпадение грейда при пересдаче не может быть близким к 1 при 4–5 уровнях и ~12 заданиях; стабильность для пользователя обеспечивает гистерезис.
- В банке пока 69 семейств (Backend 44, Frontend 41 с учётом общих модулей) и нет pretest-семейств: r_pb считается по боевым заданиям с оценкой уровня без этого задания (LOO), а не по pretest-слотам.
- Fullstack-строки (производная категория) в популяции не порождаются: каждый кандидат проходит один тест.

4. Уточнения процедуры после дымового прогона (пороги не менялись)

Вписаны после первого запуска `--quick`, до полного прогона. Ни одна метрика и ни один порог из раздела 2 не изменены; уточнены только процедуры, которые в плане были описаны неоднозначно.

1. **S2** считается на общих случайных числах: у кандидата во всех условиях один поток `s2:<id>`, меняются только истинные параметры семейств; условие σ = 0, A = 1,5 считается явно тем же способом, а не берётся из основного прогона. Иначе разница условий тонула в шуме выборки.
2. **Веса ±0,05.** Если пропорциональная компенсация выводит вес за ограничение (wF > 0,15 или wT < 0,30), этот вес остаётся на месте, а компенсацию делят остальные; сдвиг пропускается, только если и так нельзя (например, wF + 0,05).
3. **Сквозной режим подбора** повторяет путь работодателя: если парсер задал уточняющий вопрос о специализации или грейде, ответ — вариант из эталона; если парсер не нашёл вилку, работодатель вводит её в обязательной форме (значения эталона). Отдельно показан режим «только парсер, без уточнения» и режим «эталонный бриф».
4. **B2** берёт первую рекомендованную категорию, в которой есть кандидаты (для Fullstack-брифов это смежная категория Backend или Frontend), и сортирует её по точечной оценке уровня.
5. **Набор пар для жюри** — пул из объединения топ-20 систем «наш сквозной», B0, B1, B2, B3 и 10 случайных кандидатов (а не топ-50 всех систем): иначе файл весит ~8 МБ. Для P@10 и P@5 глубины 20 достаточно.
6. **Коллизии ключей** для детектора «чужой ключ» измеряются на 200 экземплярах каждого семейства Backend (сценарий утечек — Backend).
7. **Длина теста**: «показано вместе с pretest» считается по основной части (DECISIONS п.17), доборы — отдельно.
8. Добавлена **описательная диагностика без порогов**: причины нерелевантных мест в топ-10 (наш подбор и B2) и разбивка контрфакта объяснений по компонентам. Она помогает читать результат и ни на что не влияет.
9. Сетка порога CUSUM расширена до h ∈ {4…10} (справочно): порог плана по-прежнему проверяется при прод-значении h = 6.
10. V-CAT дополнен подсказкой владельцу конфигурации опроса: сетка «вес технологий × бонус за выбор», подобранная на чётных профилях и проверенная на нечётных. В прод-конфиг она не вносится и на пороги не влияет.

5. Уточнения после независимого ревью харнесса (после полного прогона; пороги не менялись)

Вписаны после полного прогона по замечаниям ревьюера. Метрики и пороги раздела 2 не изменены; каждое уточнение делает оценку строже или честнее, а не выгоднее.

1. **Утечка ответов, плитка «коалиция 20»** — берётся худшая для нас из двух стратегий списывающего (наивный ввод чужого ключа или осторожный честный ответ), а не наивная: наивный почти всегда проваливается, поэтому его доля ложных Middle занижает риск.
2. **Близнецы по тексту.** id экземпляров в симуляции уникальны по построению выдачи, поэтому «пересечение экземпляров = 0» — свойство выдачи, а не измерение. Дополнительно измеряется вероятность, что два разных экземпляра семейства совпадают по тексту (200 экземпляров каждого семейства Backend), и в модели наивного списывающего слитый ключ верен с этой вероятностью (плюс угадывание c).
3. **Экспозиция без ФСП**: децили T и места считаются по всем кандидатам рекомендованных категорий брифа, а не по ≤ 500 строкам версии подборки (обрезка по Score смещала нижние децили).
4. Добавлена описательная метрика без порога из концепции §7.2, которой не было в плане: точность и полнота тегов ✓ модулей против истинного уровня модуля. Ориентир концепции (≥ 0,90 / ≥ 0,75) показывается справочно.

6. Добавление по ответам организаторов: test-retest и разделяющая способность по специализациям

Организаторы описали ожидаемую логику: **test-retest** (один профиль проходит тест дважды, сравниваются присвоенные грейды) и **разделяющая способность** заданий (различать соседние грейды); симуляция на синтетике допустима, минимального объёма выборки нет, эталон разметки команда задаёт сама. Протокол дополнен отдельным экспериментом (`eval/spec_validation.py` , результаты — VALIDATION.md, 2.11); основные пороги из раздела 2 не менялись.

Что	Как считается
Профили	по 400 латентных профилей на каждую специализацию (Backend, Frontend, QA, DevOps), поровну по четырём грейдам, θ равномерно в полосе грейда, заявка 60/30/10
Сессии	A — основная; B — независимая параллельная (другой сид → другие клоны); C — пересдача по реальным правилам (семейства A исключены, spec-testing §7.1)
Test-retest	точное совпадение грейда, ±1, взвешенный κ, r(θ), совпадение решения сдал/не сдал, доля «переворотов» через каждый порог (в том числе рядом с порогом)
Разделяющая способность	матрица ошибок 4×5, точность решения у каждого порога (всего и при
Дискриминация заданий	r_pb каждого семейства, IRT-крутизна A_eff, прирост доли верных между соседними грейдами
Робастность	чувствительность к разбросу клонов σ_inst ∈ {0; 0,15; 0,30}; парный эксперимент «слитые ключи» (прирост θ); покрытие банка двойниками
Исключение	кандидаты с заявкой ниже истинного грейда не входят в точность: тест на заявленный грейд не выдаёт больше (повышение — отдельный тест)
Воспроизводимость	`random.Random(f"{seed}:...")` ; сид по умолчанию 20261005; число процессов результат не меняет; для жюри — `eval/jury_check.py` на своих профилях

Пороги для этого эксперимента до прогона не фиксировались (ориентиры, назначенные постфактум и потому не засчитываемые как «выполнено»: точное совпадение грейда при ретесте ≥ 0,65, ±1 ≥ 0,98, точность решения у порога ≥ 0,85 по основному разделу 2.1); результаты показаны как есть, слабые места названы в выводах 2.11.